

리버싱 프로젝트2

다단계 라이선스 검증 프로그램 리버싱 및 인증 우회 분석

정명훈

목차

1.개요

1.1 프로그램 환경 설정

1.2 프로젝트 내용 요약

2.리버싱을 통한 분석 과정

2.1 디버거 감지 프로그램을 바이너리 패치를 통해 우회

2.2 암호화된 문자열을 찾기 위한 정적 분석

2.3 암호화된 문자열 해독을 위한 동적 분석

2.4 정답을 찾기 위한 정적 분석

2.5 정적분석 내용을 이용하는 동적분석을 이용해 답 찾기

2.6 정답을 바이너리 패치를 통한 우회를 통해 얻기

2.7 배운점

2.8 한계점

1.1 프로그램 환경 설정

파이썬 프로그램은 visual studiocode에서 python 3.13(64bit)로 실행되었고

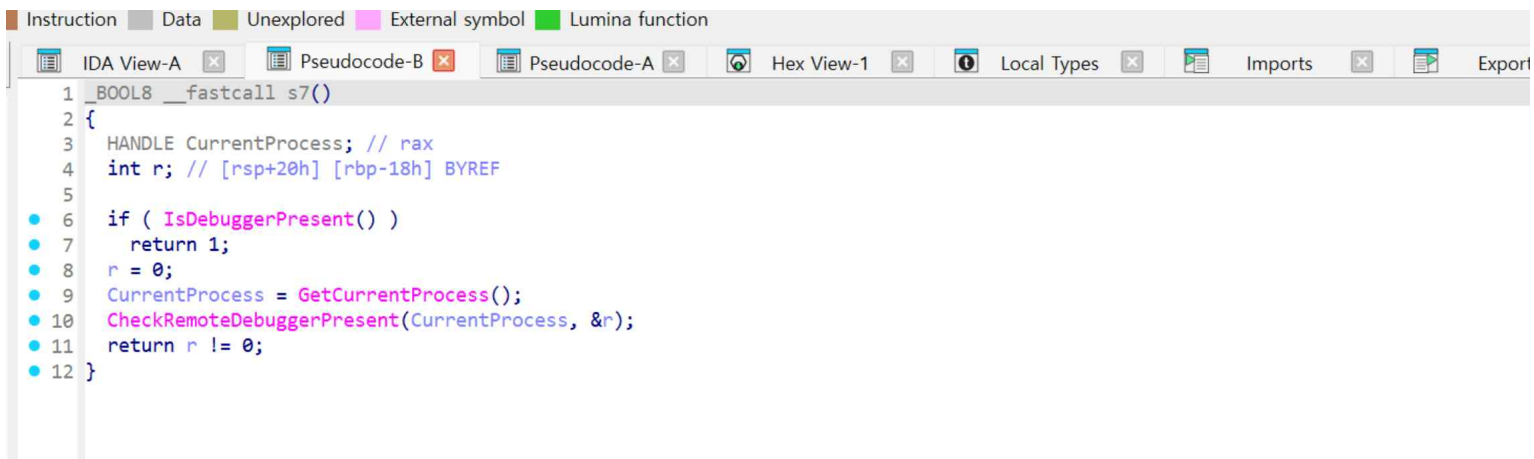
c코드는 visual studio 2022에서 release x64로 로컬 windows 디버거로 실행이 되 exe파일이 만들어졌습니다.

1.2프로젝트 내용 요약

이 프로젝트는 리버싱 프로젝트1에 이어서 더 심화된(안티디버깅 코드,암호화된 문자열들, 가짜 검증 함수,복잡한 xor, rotate,add함수들을 이용한 암호화된 답)로그인 프로그램에 대한 리버싱을 통해 분석하는 것이 목적입니다.

2.1 디버거 감지 프로그램을 바이너리 패치를 통해 우회

우선 ida에 올리고 xdbg64에서 실행하려고 하니 access denied가 계속 뜨길래 ida를 까서 분석을 해보니



```
1 _BOOL8 __fastcall s7()  
2 {  
3     HANDLE CurrentProcess; // rax  
4     int r; // [rsp+20h] [rbp-18h] BYREF  
5  
6     if ( IsDebuggerPresent() )  
7         return 1;  
8     r = 0;  
9     CurrentProcess = GetCurrentProcess();  
10    CheckRemoteDebuggerPresent(CurrentProcess, &r);  
11    return r != 0;  
12 }
```

함수s7에 들어가보니 누가봐도 이상해 보이는 IsDebuggerPresent() 함수가 있고



```
30     if ( s7() )  
31     {  
32         printf("%s", m4);  
33         return 1;  
34     }
```

s7이 참이면 프로그램이 종료된다는 걸 main부분을 통해서 알 수 있다. 따라서 디버거를 실행하기 위해서 x64dbg와 ida의 주소를 동기화시키기도 ida에서 s7함수의 위치 주소를 찾아 x64dbg로 가서 그 함수를

00007FF713BA1380	48:83EC 38	sub rsp,38	s7
00007FF713BA1384	48:8B05 753C0000	mov rax,qword ptr ds:[<_security_cook	rax:mainCRTStartup, 00007FF7
00007FF713BA138B	48:33C4	xor rax,rsp	rax:mainCRTStartup
00007FF713BA138E	48:894424 28	mov qword ptr ss:[rsp+28],rax	[rsp+28]:BaseThreadInitThunk
00007FF713BA1393	FF15 6F1C0000	call qword ptr ds:[<&IsDebuggerPresent	main.c:102

00007FF713BA1380	31C0	xor eax,eax	s7
00007FF713BA1382	C3	ret	
00007FF713BA1383	3848 8B	cmp byte ptr ds:[rax-75],cl	rax-75:__scrt_common_main_se
00007FF713BA1386	05 753C0000	add eax,3C75	
00007FF713BA138B	48:33C4	xor rax,rsp	rax:mainCRTStartup
00007FF713BA138E	48:894424 28	mov qword ptr ss:[rsp+28],rax	[rsp+28]:BaseThreadInitThunk
00007FF713BA1393	FF15 6F1C0000	call qword ptr ds:[<&IsDebuggerPresent	main.c:102

이렇게 늘 거짓이 되게 바꾸고 영구 패치를 시켜서 디버거 감지 함수를 무력화 시켰다.

```

D:\source\security\license_project\x64\Release\license_project_v2.exe
Name: dsasad
Serial: dsadsad
Wrong serial.
  
```

이제는 아까 전과 달리 입력이 제대로 이루어진다는 사실을 알 수 있다.

2.2 암호화된 문자열을 찾기 위한 정적 분석

어제에 이어서 디버거를 이용해 몇가지 입력을 넣어본 후 현재 상태로는 알아낼 게 없어 우선 정적분석을 먼저 하기 위해 ida에서 문자열 table을 shift + f12단축키를 눌러서 확인을 한다.

Function name	Address	Length	Type	String
__local_stdio_printf_options	.rdata:00007FF...	00000006	C	%s:%s
__local_stdio_scanf_options	.rdata:00007FF...	00000014	C	%04X-%04X-%04X-%04X
printf	.rdata:00007FF...	00000043	C - U...	D:\source\security\license_project\x64\Release\license_project.pdb
scanf	.rdata:00007FF...	00000009	C	.text\$mn
snprintf	.rdata:00007FF...	0000000C	C	.text\$mn\$00
s0	.rdata:00007FF...	00000008	C	.text\$x
s1	.rdata:00007FF...	00000009	C	.idata\$5
s2	.rdata:00007FF...	00000007	C	.00cfg
s3	.rdata:00007FF...	00000009	C	.CRT\$XCA
s4	.rdata:00007FF...	0000000A	C	.CRT\$XCAA
s5	.rdata:00007FF...	00000009	C	.CRT\$XCZ
s6	.rdata:00007FF...	00000009	C	.CRT\$XIA
s7	.rdata:00007FF...	0000000A	C	.CRT\$XIAA
s9	.rdata:00007FF...	0000000A	C	.CRT\$XIAC
t0	.rdata:00007FF...	00000009	C	.CRT\$XIZ
main	.rdata:00007FF...	00000009	C	.CRT\$XPA
_security_check_cookie	.rdata:00007FF...	00000009	C	.CRT\$XPZ
pre_c_initialization	.rdata:00007FF...	00000009	C	.CRT\$XTA
	.rdata:00007FF...	00000009	C	.CRT\$XTZ

그러면 이런식으로 뜨는데 확실히 프로그램이 암호화가 되서 이전보다 유용한 문자열을 찾기가 어려워졌다. 그 다음에는 함수들도 보면 main과 printf,scanf와 같은 함수를 제외한 나머지 함수들은 s0-s9,t0 이런식으로 이름이 지어져서 그 의미를 찾기가 이전보다 어려워졌다는 걸 알 수 있다. 다행이도 main은 암호화가 되지 않아서 main의 위치는 쉽게 찾을 수가 있었고

```

26 s0(p1, 8, m1);
27 s0(p2, 9, m2);
28 s0(p3, 14, m3);
29 s0(p4, 15, m4);
30 if ( s7() )
31 {
32     printf("%s", m4);
33     return 1;
34 }
35 else
36 {
37     printf("%s", m0);
38     scanf("%63s", name);
39     printf("%s", m1);
40     scanf("%63s", input_serial);
41     v4 = t0(name, input_serial);
42     v5 = m3;
43     if ( v4 )
44         v5 = m2;
45     printf("%s", v5);
46     return 0;
47 }
48 }

```

그 다음에는 main에서 입력을 받는 scanf와 이때 입력값을 저장하는 input_serial과 name 변수를 찾아냈다. 그 다음에 이제 복호화 함수를 찾아야 한다. 복호화 함수 찾기는 그리 어렵지 않은데 memcpy된 문자열들이 다 s0을 거치는 걸 보니 s0이 복호화 함수라는 것을 쉽게 알 수 있다.

```

8  *(_DWORD *)p0 = 1060584212;
9  *(_WORD *)&p0[4] = 31328;
10 memcpy(p1, "\t?(3;6`z", sizeof(p1));
11 *(_DWORD *)p2 = 673723673;
12 memcpy(&p2[4], "?9.{P", 5);
13 memcpy(p3, "\r(54=z)?(3;6tP", 14);
14 memcpy(p4, "\x1B99?)z>?43?>tP", 15);
15 s0(p0, 6, m0);

```

그 다음에는 encoded data는 이렇게 내놓고 ida에 memcpy로 나와 있기 때문에 쉽게 찾을 수 있고 so내부의 매커니즘을 ai의 도움을 일부 받아 추론해 보니

```

5  *((__m128 *)v6 - 2) = _mm_xor_ps(v11, s1128);
6  }
7  while ( (__int64)&v6[-16LL - (_QWORD)out] < (__int64)(n & 0xFFFFFFFFFFFFFFFFC0uLL) );
8  }
9  v12 = v3;
10 v13 = n;
11 if ( v12 < n )
12 {
13     v14 = p - (unsigned __int8 *)out;
14     v15 = &out[v12];
15     v16 = n - v12;
16     do
17     {
18         v17 = (v15++)[v14];
19         *(v15 - 1) = v17 ^ 0x5A;
20         --v16;
21     }
22     while ( v16 );
23 }
24 out[v13] = 0;
25 }

```

암호화 과정이

```
encode(p2, n, m2);
```

```
for (int i = 0; i < n; i++) {
```

```
    m2[i] = p2[i] ^ 0x5A;
```

```

}

m2[n] = '\0';과 같은 형태로 이루어져 있고 이를 통해 복호화 반복문 분석하면

def decode(p):

```

return bytes([b ^ 0x5A for b in p]).decode(errors="ignore") 이런 형태의 함수가 나온다는 사실을 알 수 있다. 복호화 알고리즘을 Python으로 재구현해서 실행시키면

```

def dec(data):

    return bytes(b ^ 0x5A for b in data)

p0 = (1060584212).to_bytes(4, "little") + (31328).to_bytes(2, "little")

p1 = b"Wt?(3;6`z"

p2 = (673723673).to_bytes(4, "little") + b"?9.{P"

p3 = b"Wt(54=z)?(3;6tP"

p4 = b"Wx1B99?))z>?43?>tP"

for i, p in enumerate([p0, p1, p2, p3, p4]):

    print(i, dec(p))

```

출력 결과

```

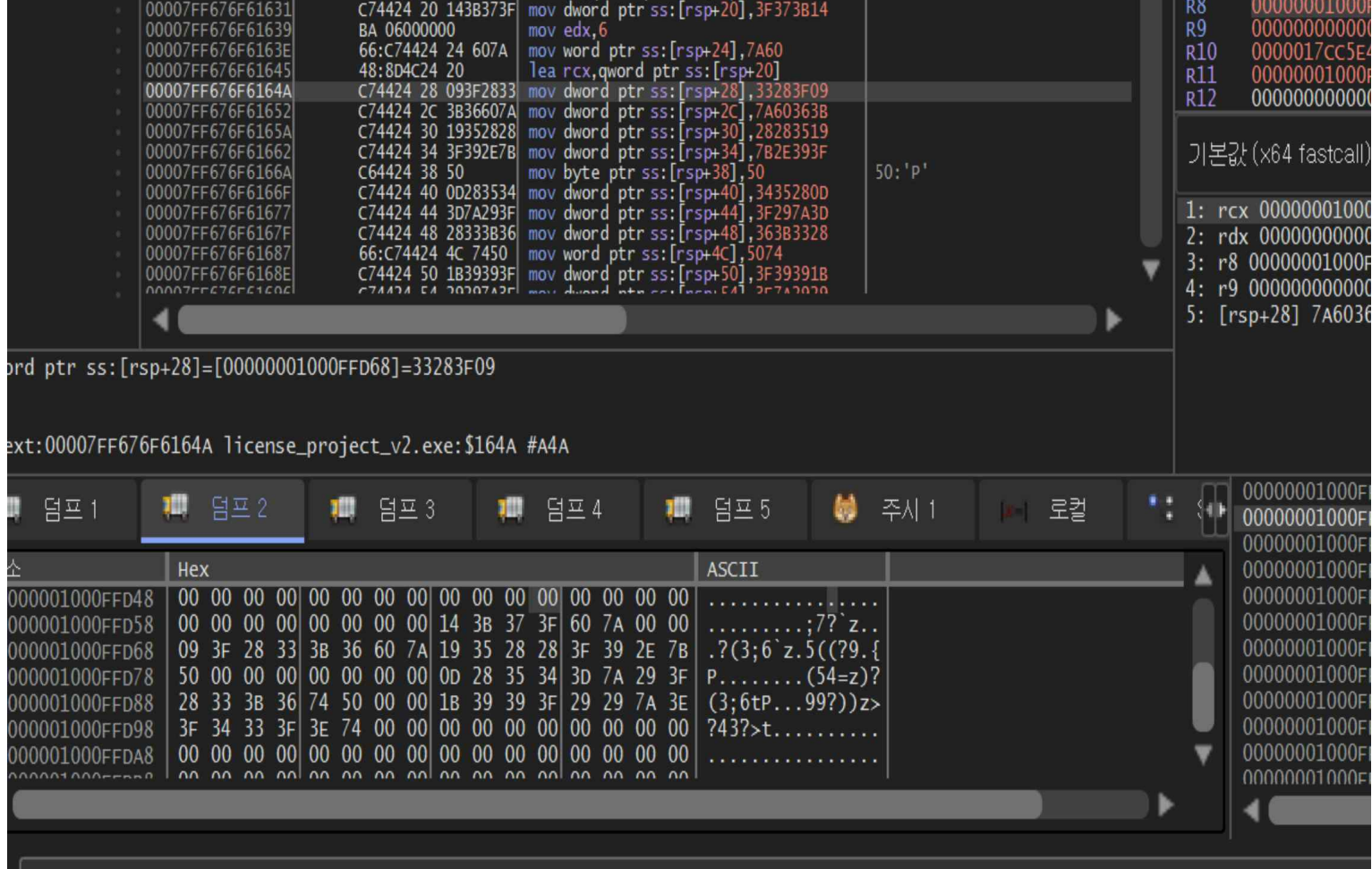
p0 b'Name: '
p1 b'Serial: '
p2 b'Correct!\n'
p3 b'Wrong serial.\n'
p4 b'Access denied.\n'

```

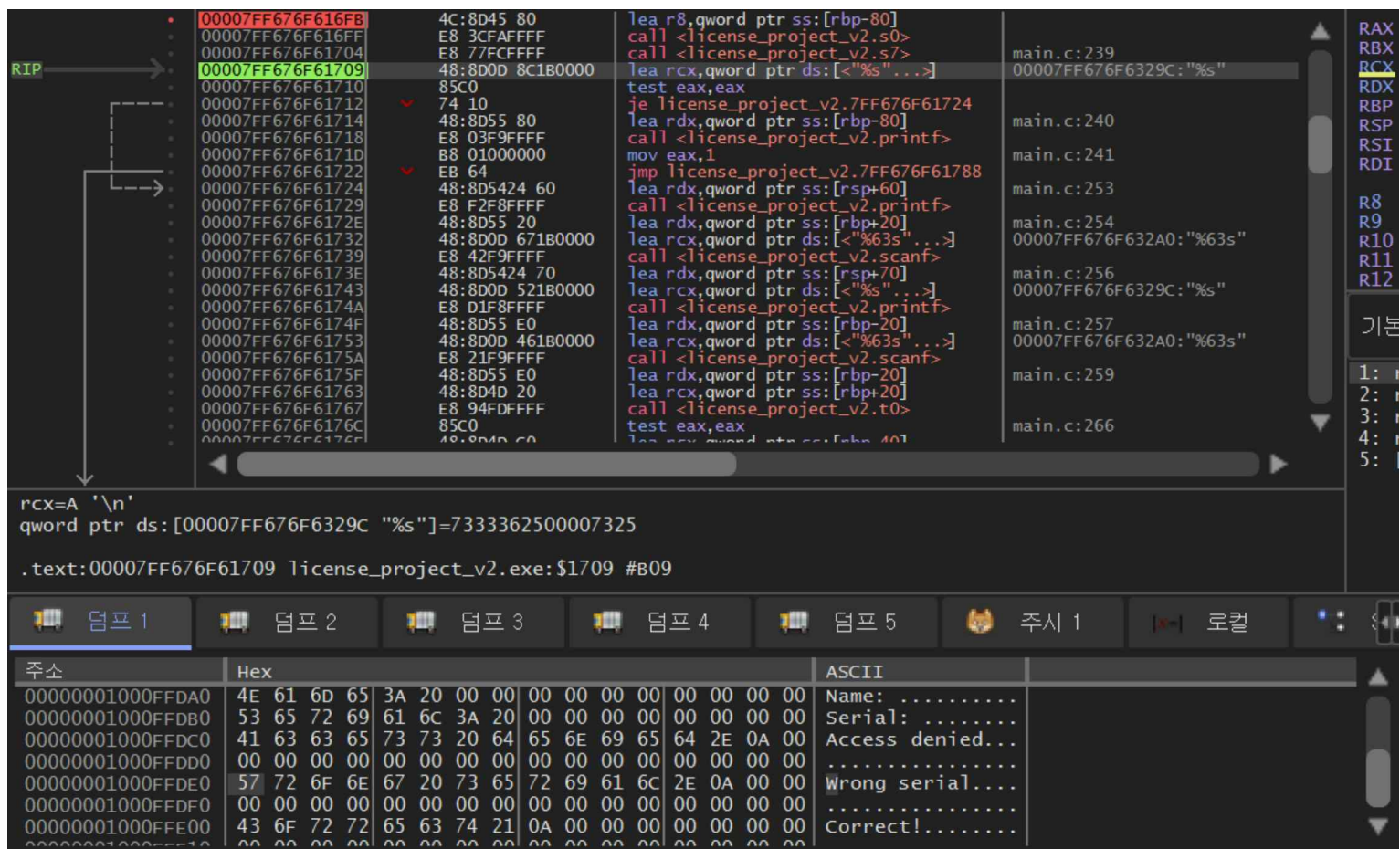
가 나오고 이를 ida의 코드와 대조해 분석하면 if(v4)인 경우에 출력되는 게 correct임으로 v4인 경우 정답이라는 사실을 알 수가 있다.

2.3 암호화된 문자열 해독을 위한 동적 분석

자 이제 동적 분석으로도 한 번 암호화되기 전 후를 검증해보자



복호화되기 이전에는 이런 상태이고 복호화가 된 이후에는



이런 상태가 된다. 이런 방식으로 ida에서 복호화 함수를 추론후 실행 전후 결과를 비교해서 복호화된 값을 쉽게 얻을 수 있다. 지금은 프로그램이 단일이고 간단한 편이라 정적분석이 더 나은 선택지로 보이지만 이러한 동적분석은 프로그램이 복잡해질수록 더 많이 쓰일거다.

2.4정답을 찾기 위한 정적 분석

그 다음 과제는 Name 입력값이 어디에 저장되는지 확인하면 name입력값과 serial입력값은 ida기준으로 각각 name과 input_serial에 저장이 된다

해시 계산 루틴 추적해 보면 간단하게 to함수가 사실상 해시계산을 하는 함수임을 알 수 있고 이 함수를 분석하는 게 정답을 찾기 위한 가장 핵심 부분이다.

하지만 역시 이전 프로젝트보다 리버싱 난이도가 올라가서 그런지 to내에 다양한 서브루틴 함수들이 있고 이들은 꽤나 복잡하다. 일단 내부함수로 s1,s3,s4,s5,s6,s9등이 있고 이들을 하나씩 자세히 분석해야 한다.

우선 s3를 보니 이건 tolower가 있는 걸 보니 그냥 이름을 입력으로 받아 대문자를 소문자로 바꿔주는 함수라는 걸 쉽게 알 수 있다. 따라서 동적분석은 그냥 생략하겠다.

s4의 경우에는 XOR,Rotate(ROL/ROR),덧셈을 섞어서 내부 상태를 만들고 마지막 상태의 하위 7 비트가 49인지 검사하는 검증함수라고 봐야 한다

.s1은 hash = 2166136261

for each byte in input:

```
hash = (hash ^ byte) * 16777619
```

와 같이 입력 문자열을 32비트 해시값으로 바꿔주는 함수라고 할 수 있다.

s2함수는 입력x를 오른쪽으로 25비트 회전(Rotate Right)시키는 함수다

s6함수는

```
BOOL8 s6(const char *a, const char *b)
```

```
{
```

```
uint32_t x = s2(s1(a) ^ 0x6D2B79F5);
```

```
uint32_t y = s2(s1(b) ^ 0x31C3A59B);
```

```
return ((x ^ y) & 3) == 1;
```

}로 a, b 두 문자열의 해시값을 각각 변형한 뒤, 두 결과의 하위 2비트 관계만 검사하는 함수다.

s9함수는

```
void s9(const char *name, char *out)
```

```
{  
  
    char e0[14];  
  
    char t[32];  
  
    char buf[128];  
  
    // e0 = 암호화된 14바이트  
  
    s0(e0, 14, t);                // t에 복호화 문자열 저장  
  
    snprintf(buf, 0x80, "%s:%s", name, t);  
  
    uint32_t h = s1(buf);  
  
    uint32_t part1 = h & 0xFFFF;  
  
    uint32_t part2 = (h ^ 0xA5C3F19B) & 0xFFFF;  
  
  
    uint32_t y = s2(h) + 0x13579BDF;  
  
    uint32_t part3 = y & 0xFFFF;  
  
  
    uint32_t part4 = (y ^ (h ^ 0xA5C3F19B)) & 0xFFFF;  
  
  
    snprintf(out, 0x14, "%04X-%04X-%04X-%04X",  
             part1, part2, part3, part4);  
  
}
```

대략적으로 이런 느낌이다. 그리고 실제 to함수내 검증 메커니즘을 살펴보다 보니 s4,s6함수일 때 그저 s1함수를 호출하고 끝나는 걸로 봐서 이 함수들은 분석 방해용 더미 검증함수일 가능성이 높다.그리고 사용자가 입력한 serial code와 name을 기반으로 해서 s3,s9을 이용해 만든 serial값과 일치여부를 통해 참과 거짓을 판별하는 프로그래밍을 알 수 있다.

그래서 가장 중요한 serial을 생성하는 함수인 s9에서 추가로 분석을 해보면 t에 들어갈 복호화 된 문자열은

```
mov dword ptr [rsp+108h+e0], 680A0C08h
```



```
mov dword ptr [rsp+108h+e0+4], 161B0905h
```

```
mov dword ptr [rsp+108h+e0+8], 6A68050Eh
```

```
mov word ptr [rsp+108h+e0+0Ch], 6C68h
```

인 e0를 복호화를 한 것인데 이는 리틀 엔디안으로 나타낸 것이기 때문에 e0값은 실질적으로는 08 0C 0A 68 05 09 1B 16 0E 05 68 6A 68 6C가 되고 이를 s0의 복호화 함수에 집어 넣으면

```
def dec(data):
    return bytes(b ^ 0x5A for b in data)

# p0 = (1060584212).to_bytes(4, "little") + (31328).to_bytes(2, "little")
# p1 = b"\t?(3;6`z"
# p2 = (673723673).to_bytes(4, "little") + b"?9.{P"
# p3 = b"\r(54=z)?(3;6tP"
# p4 = b"\x1B99?)z>?43?>tP"

#for i, p in enumerate([p0, p1, p2, p3, p4]):
#    print(i, dec(p))

eo = "08 0C 0A 68 05 09 1B 16 0E 05 68 6A 68 6C"
eo = bytes.fromhex(eo)
print(dec(eo).decode(errors="ignore"))
```

```
PS C:\Users\정명훈\AppData\Local\Programs\Microsoft VS Code> & C:\Users\정명훈\AppData\Local\Programs\Python\Python313\python.exe c:/Users/정명훈/OneDrive/Desktop/security_project/reverse_proj2.py
RVP2_SALT_2026
```

이와 같은 결과가 나온다 이를 통해 t = RVP2_SALT_2026라는 걸 알 수 있고 그 다음 과정을 보면 `snprintf(buf, 0x80, "%s:%s", name, t);`을 보면 name값은 위에서 입력받은 이름을 위에서 `s3(a);`로 lowercase로 받은 값이고 buf는 `lower_case_name:RVP2_SALT_2026`라는 걸 알 수 있다. 그 다음에는 이 buf값을 FNV-1a 32비트 해시 함수 s1에 넣어주면 그 값을 얻을 수 있다 그 다음에는 그렇게 얻은 h값을 뭘 하는데 디컴파일러가 출력한 결과가 정확하지 않아서 어셈블리어를 보면

```
mov     r9d, eax
```

```
mov     ecx, eax          ; x
```

```
xor     r9d, 0A5C3F19Bh
```

```
mov     r11d, eax
```

```
call    s2
```

여기서 s1의 반환값 eax가 해시값 h이고

```
r9d = h ^ 0xA5C3F19B;
```

```
r11d = h;
```

s2(h);

즉

$x = h \wedge 0xA5C3F19B$;

rot = s2(h); 의 의미를 가지고

s2는 ROR32(h, 25) 즉 회전시키는 함수이기 때문에 32비트 기준으로는 rot = ROL32(h, 7);와 같은 의미를 지닌다.

그 다음에 y를 계산하면 lea edx, [rax+13579BDFh] 는 s2(h)의 결과가 rax/eax에 들어있고, 거기에 0x13579BDF를 더한다는 의미로 32비트 c언어로 표현시 $y = (s2(h) + 0x13579BDF) \& 0xFFFFFFFF$;가 된다.

그 다음에는 마지막으로 snprintf에 들어가는 값을 보면 된다.

```
movzx    r8d, dx
mov      eax, edx
xor      eax, r9d
movzx    r10d, ax
mov      [rsp+108h+var_D8], r10d
mov      [rsp+108h+var_E0], r8d
lea      r8, a04x04x04x04x ; "%04X-%04X-%04X-%04X"
movzx    eax, r9w
movzx    r9d, r11w
mov      dword ptr [rsp+108h+var_E8], eax
call     snprintf
```

이걸 하나씩 해석해보면

1번째 파트는 movzx r9d, r11w = part1 = h & 0xFFFF; 이고

2번째 파트는 movzx eax, r9w, mov [rsp+var_E8], eax = part2 = (h ^ 0xA5C3F19B) & 0xFFFF; 와 같은 의미이다.

3번째 파트는 movzx r8d, dx mov [rsp+var_E0], r8d = part3 = y & 0xFFFF;

4번째 파트는 mov eax, edx xor eax, r9d movzx r10d, ax mov [rsp+var_D8], r10d = eax = y;

$\text{eax} \wedge = \text{h} \wedge 0\text{xA5C3F19B}$; 와 같은 의미를 가지고 이는 $\text{part4} = (\text{y} \wedge (\text{h} \wedge 0\text{xA5C3F19B})) \& 0\text{xFFFF}$; 이와 동일한 의미이다.

그리고 최종 `sprintf`명령어는

```
mov     name, rdi
```

```
mov     edx, 14h
```

```
lea     r8, a04x04x04x04x ; "%04X-%04X-%04X-%04X"
```

```
call    snprintf 이고
```

이는 c코드로 번역하면

```
snprintf(out, 0x14, "%04X-%04X-%04X-%04X",  
         part1, part2, part3, part4);
```

따라서 최종적으로 시리얼 생성 알고리즘은

```
name = lowercase(name);
```

```
secret = "RVP2_SALT_2026";
```

```
buf = name + ":" + secret;
```

```
h = FNV1a32(buf);
```

```
x = h ^ 0xA5C3F19B;
```

```
y = ROR32(h, 25) + 0x13579BDF;
```

```
part1 = h & 0xFFFF;
```

```
part2 = x & 0xFFFF;
```

```
part3 = y & 0xFFFF;
```

```
part4 = (y ^ x) & 0xFFFF;
```

```
serial = "%04X-%04X-%04X-%04X";
```

다음과 같고 이를 파이썬을 이용해서 구현하면

```
def fnv1a_32(data: bytes) -> int:
```

```
h = 2166136261
```

```
for b in data:
```

```
    h ^= b
```

```
    h = (h * 16777619) & 0xFFFFFFFF
```

```
return h
```

```
def ror32(x: int, n: int) -> int:
```

```
    x &= 0xFFFFFFFF
```

```
    return ((x >> n) | (x << (32 - n))) & 0xFFFFFFFF
```

```
def make_serial(name: str) -> str:
```

```
    name = name.lower()
```

```
    secret = "RVP2_SALT_2026"
```

```
    buf = f"{name}:{secret}".encode("ascii")
```

```
    h = fnv1a_32(buf)
```

```
    x = (h ^ 0xA5C3F19B) & 0xFFFFFFFF
```

```
    y = (ror32(h, 25) + 0x13579BDF) & 0xFFFFFFFF
```

```
    part1 = h & 0xFFFF
```

```
    part2 = x & 0xFFFF
```

```
    part3 = y & 0xFFFF
```

```
    part4 = (y ^ x) & 0xFFFF
```

```
    return f"{part1:04X}-{part2:04X}-{part3:04X}-{part4:04X}"
```

```
print(make_serial("Alice"))
```

이렇게 된다 앞으로는 name에 아무값이나 입력하고 파이썬을 이용해서 그 name에 맞는 serial

number를 입력하면 로그인 할 수 있고 아래 그 결과가 나와 있다.

```
D:\source\security\license_project\x64\Release\license_project_v2.exe
Name: Alice
Serial: 1405-E54E-0689-E3C7
Correct!
```

2.5정적분석 내용을 이용하는 동적분석을 이용해 답 찾기

이제 정적 분석을 끝냈으니 동적 분석으로 넘어가자 동적분석을 위해 늘 그랬듯이 ida와 xdbg 모듈의 주소를 맞춰주고 그 다음에 우선 ida의 checker함수의 주소를 찾아서 ctrl + g로 이동하

00007FF70AFD14FD	CC	int3	
00007FF70AFD14FE	CC	int3	
00007FF70AFD14FF	CC	int3	
00007FF70AFD1500	48:895C24 18	mov qword ptr ss:[rsp+18],rbx	checker
00007FF70AFD1505	48:896C24 20	mov qword ptr ss:[rsp+20],rbp	
00007FF70AFD150A	56	push rsi	rsi:"minkernel\\ldr\\ldrinit
00007FF70AFD150B	57	push rdi	
00007FF70AFD150C	41:56	push r14	
00007FF70AFD150E	48:83EC 50	sub rsp,50	
00007FF70AFD1512	48:8B05 E73A0000	mov rax,qword ptr ds:[<_security_cook	
00007FF70AFD1519	48:33C4	xor rax,rsp	
00007FF70AFD151C	48:894424 48	mov qword ptr ss:[rsp+48],rax	[rsp+48]:"minkernel\\ldr\\ld
00007FF70AFD1521	48:8BDA	mov rbx,rdx	rbx:"LdrpInitializeProcess"
00007FF70AFD1524	4C:8BF1	mov r14,rcx	rcx:NtQueryInformationThread
00007FF70AFD1527	E8 34FDFFFF	call <license_project_v2.s3>	main.c:163
00007FF70AFD152C	49:8BCE	mov rcx,r14	main.c:165, rcx:NtQueryInfor
00007FF70AFD1537	E8 5CFDFFFF	call <license_project_v2.s4>	

고 breakpoint를 걸고 checker함수 내부 함수를 거치면서 레지스터값의 변화를 확인하기 위해 서 각 함수별로 breakpoint를 걸었다.

```
D:\source\security\license_project\x64\Release\license_project_v2.exe
Name: Tom
Serial: 03E1-F27A-8CDD-7EA6
```

그 다음에는 이름과 그에 따른 serial번호를 입력하고 f9를 눌러가면서 동적 분석을 시작했다.

00007FF70AFD1500	48:895C24 18	mov qword ptr ss:[rsp+18],rbx	checker	RAX	00006940D6CA4C90	
00007FF70AFD1505	48:896C24 20	mov qword ptr ss:[rsp+20],rbp		RBX	00000000DB4FFC80	"03E1-F27A-8CDD-7EA6"
00007FF70AFD150A	56	push rsi		RCX	00000000DB4FFC80	"Tom"
00007FF70AFD150B	57	push rdi	rdi:&"ALLUSERSPROFILE=C:\\Pr	RDX	00000000DB4FFC80	"03E1-F27A-8CDD-7EA6"
00007FF70AFD150C	41:56	push r14	r14:"Tom"	RBP	00000000DB4FFCA0	
00007FF70AFD150E	48:83EC 50	sub rsp,50		RSP	00000000DB4FFB30	&"%0\$"
00007FF70AFD1512	48:8B05 E73A0000	mov rax,qword ptr ds:[<_security_cook		RSI	0000000000000000	
00007FF70AFD1519	48:33C4	xor rax,rsp		RDI	000002CF0D16F000	&"ALLUSERSPROFILE=C:\\ProgramData"
00007FF70AFD151C	48:894424 48	mov qword ptr ss:[rsp+48],rax		R8	7FFFFFFFFFFFFFFF	
00007FF70AFD1521	48:8BDA	mov rbx,rdx	rbx:"03E1-F27A-8CDD-7EA6", r	R9	000002CF0D16BCA0	
00007FF70AFD1524	4C:8BF1	mov r14,rcx	r14:"Tom", rcx:"Tom"	R10	000002CF0D16BCA0	
00007FF70AFD1527	E8 34FDFFFF	call <license_project_v2.s3>	main.c:163	R11	00000000DB4FFFA0	&"%0\$"
00007FF70AFD152C	49:8BCE	mov rcx,r14	main.c:165, rcx:"Tom", r14:"	R12	0000000000000000	
00007FF70AFD152F	E8 5CFDFFFF	call <license_project_v2.s4>				
00007FF70AFD1534	48:8BCB	mov rcx,rbx	main.c:166, rcx:"Tom", rbx:"			
00007FF70AFD1537	8BE8	mov ebp,eax				

위 화면은 함수 s3을 실행하기 이전의 레지스터값의 스크린샷이다.

00007FF70AFD1524	4C:8BF1	mov r14,rcx	
00007FF70AFD1527	E8 34FDFFFF	call <license_project_v2.s3>	
00007FF70AFD152C	49:8BCE	mov rcx,r14	
00007FF70AFD152F	E8 5CFDFFFF	call <license_project_v2.s4>	

RAX	0000000000000000	
RBX	000000D0DB4FFC80	"03E1-F27A-8CDD-7EA6"
RCX	000000D0DB4FFC80	"tom"
RDX	000000D0DB4FFC80	"03E1-F27A-8CDD-7EA6"
RBP	000000D0DB4FFCA0	
RSP	000000D0DB4FFB30	&"%03E1"
RSI	0000000000000000	
RDI	000002CF0D16F000	&"ALLUSERSPROFILE=C:\\ProgramData"

위 화면은 s3을 실행한 후의 레지스터값의 스크린샷으로 RCX에 저장된 Tom값이 tom으로 바뀐것을 보아 s3은 이름을 전부 소문자로 바꿔주는 함수라는 사실을 추론할 수 있다.

그 다음에는 s4를 실행해보면

RCX	000000D0DB4FFC80	"03E1-F27A-8CDD-7EA6"
R12	0000000000000000	
R13	0000000000000000	
R14	000000D0DB4FFCC0	"tom"
R15	0000000000000000	

rcx에 내가 입력한 serial값이 들어가 있고 r14에 이름이 저장된 걸 제외하고는 딱히 레지스터 값이 변하지는 않았다. ida에서 정적분석한 결과를 보면 그냥 입력한 값을 회전시킨 후에 하위 7비트가 일치하는지 여부에 따라 0,1을 반환하고 실제로 영향을 미치는 함수는 아닌 거 같아 그냥 넘어갔다.

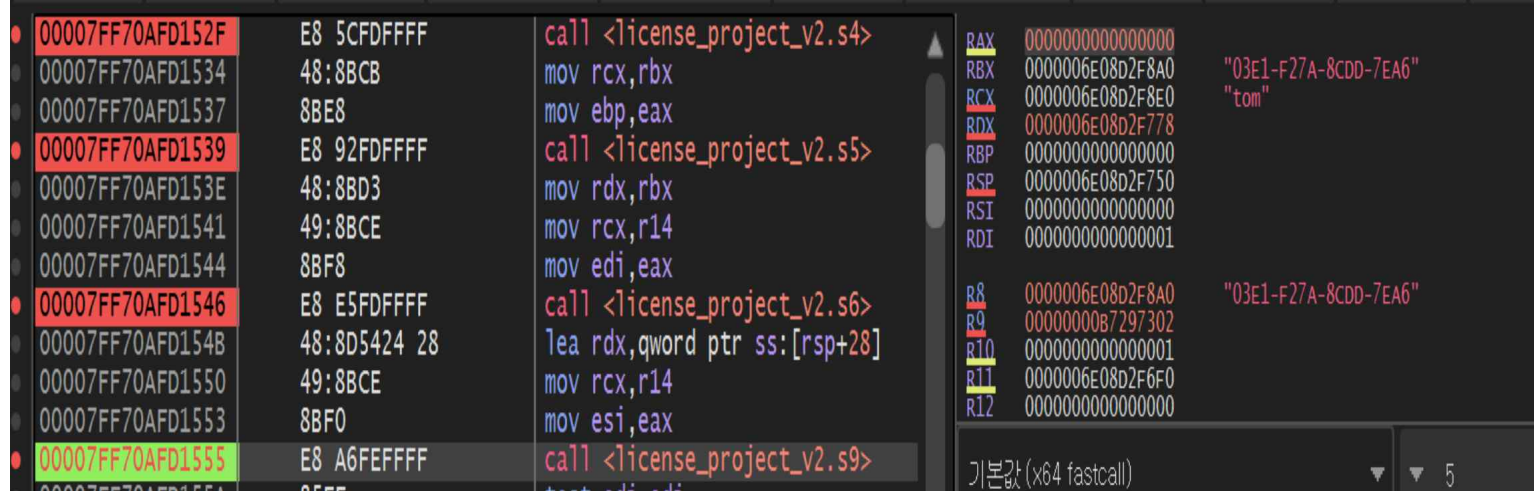
그 다음에 s5을 실행해보면

00007FF70AFD1539	E8 92FDFFFF	mov ecx,ecx
00007FF70AFD153E	48:8BD3	call <license_project_v2.s5>
00007FF70AFD1541	49:8BCE	mov rdx,rbx
00007FF70AFD1544	8BF8	mov rcx,r14
00007FF70AFD1546	E8 E5FDFFFF	mov edi,eax
		call <license_project_v2.s6>

RAX	0000000000000001	
RBX	0000006E08D2F8A0	"03E1-F27A-8CDD-7EA6"
RCX	0000006E08D2F8E0	"tom"
RDX	0000006E08D2F8A0	"03E1-F27A-8CDD-7EA6"
RBP	0000000000000000	

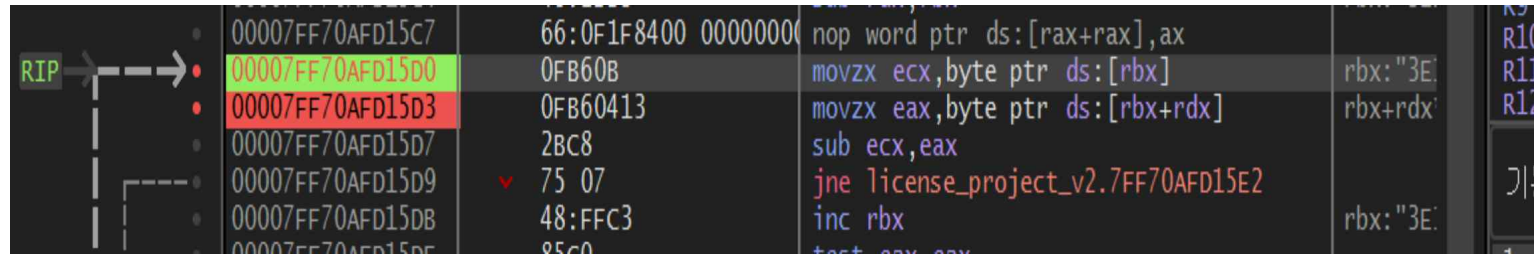
우선 정적분석으로 ida에서는 s5의 반환값이 무조건 참이어야 checker가 참을 반환한다는 사실과 checker가 serial값을 입력으로 받아서 주어진 형식에 맞는지 확인하는 함수라는 사실을 이용하면 내가 지금 입력한 serial값 xxxx-xxxx-xxxx-xxxx꼴 형태로 입력해야지만 로그인이 가능하다는 걸 알 수 있었고 실제로 동적분석으로 rax 레지스터를 보면 1즉 참을 반환했다.

그 다음에는 s6을 실행하면

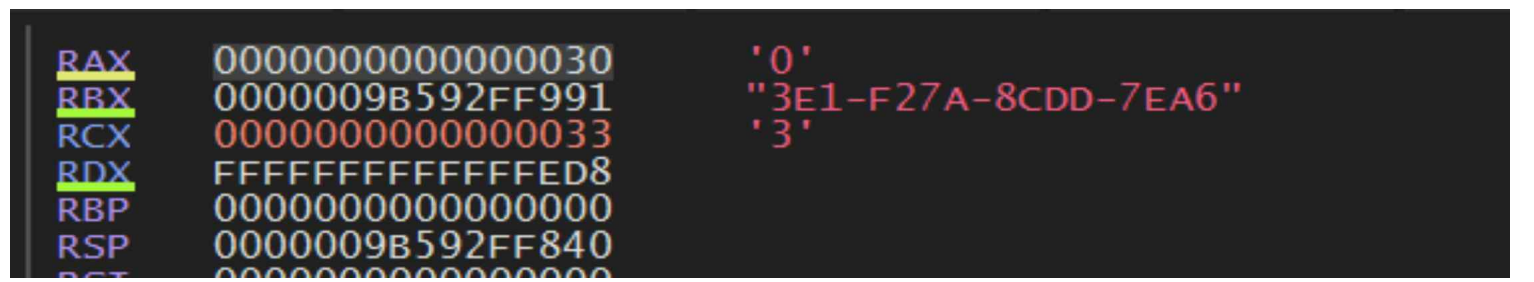


rdx에 있던 serial값만 사라지고 나머지는 전부 그대로 존재하며 정적분석 결과 사실 s6도 별 의미없는 덤프함수이고 단지 참 거짓 여부를 반환하는 함수인데 rax = 0 즉 거짓이긴 한데 정적분석 상 참 거짓이 의미가 없기 때문에 그냥 넘어간다.

그 다음에 s9함수를 실행하면

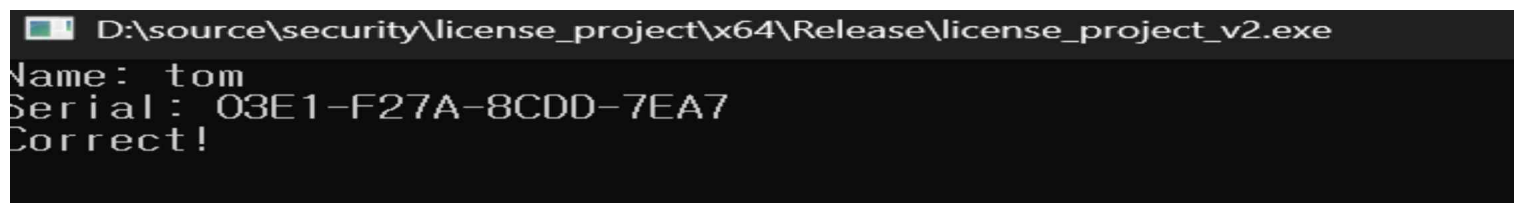


이렇게 여기로 오고 정적분석으로 s9가 입력된 이름을 기반으로 serial을 만드는 함수라는 사실과 이 부분의 어셈블리 부분이 내가 입력한 serial값과 실제 s9가 내 이름을 기반으로 만든 serial값을 비교하는 과정이라는 걸 쉽게 알 수 있고



주소	Hex	ASCII
0000009B592FF868	30 33 45 31 20 46 32 37 41 20 38 43 44 44 20 37	03E1-F27A-8CDD-7

위의 movzx부분 dml ds:[rbx + rdx]부분의 값을 덤프 따라가기로 보면 s9로 만들어낸 serial값이 나와있는 걸 확인할 수 있다. 결국 다음 입력때는 내가 이름을 그대로 입력하고 아래 나온 serial값을 그대로 입력하면 문제 해결이다.



사실 나는 간단하게 그냥 이전의 중간과정은 무시하고 이 부분 레지스터만 확인하면 안되나라는 생각도 했지만

현재 방식	어려운 문제에서 막히는 이유
정답 serial 레지스터 따라가기	정답이 평문으로 안 만들어짐
덤프에서 문자열 보기	내부 값이 바이너리/암호문/해시일 수 있음
비교 직전 값 확인	비교 함수가 직접 구현되거나 숨겨짐
한 번 실행해서 답 얻기	안티디버깅/타이밍/가짜 경로 가능
정답 문자열 복사	서버 검증이면 로컬에 정답 없음
특정 name의 serial 얻기	keygen 제작에는 알고리즘 이해가 필요

이러한 이유 때문에 동적분석을 할 때에도 중간 과정을 하나씩 검증해야 한다는 사실을 알게 되었다.

2.6 정답을 바이너리 패치를 통한 우회를 통해 얻기

바이너리 패치를 통한 우회를 하려고 하는 데 사실 이 프로그램의 경우에는 우회할 수 있는 방법이 되게 많은데 예를 들면



```
26 s0(p1, 8, m1);
27 s0(p2, 9, m2);
28 s0(p3, 14, m3);
29 s0(p4, 15, m4);
30 if ( s7() )
31 {
32     printf("%s", m4);
33     return 1;
34 }
35 else
36 {
37     printf("%s", m0);
38     scanf("%63s", name);
39     printf("%s", m1);
40     scanf("%63s", input_serial);
41     v4 = checker(name, input_serial);
42     v5 = m3;
43     if ( v4 )
44         v5 = m2;
45     printf("%s", v5);
46     return 0;
47 }
48 }
```

이 스크린샷에서 v4가 참일 때만 정답이 나오는데 1번째 방법으로는 그냥 checker함수를 늘 1을 반환하게 패치를 해도 되고 2번째 방법으로는 printf("%s", v5)을 printf("%s", m2)로 바꿔도 된다.

우선 먼저 1번째 방법을 사용해 보면

00007FF70AFD14FF	CC	int3	
00007FF70AFD1500	48:895C24 18	mov qword ptr ss:[rsp+18],rbx	checker
00007FF70AFD1505	48:896C24 20	mov qword ptr ss:[rsp+20],rbp	
00007FF70AFD150A	56	push rsi	rsi:"mi
00007FF70AFD150B	57	push rdi	
00007FF70AFD150C	41:56	push r14	

여기에서 위의 스크린샷 부분의 내용을

00007FF70AFD14FF	CC	int3	
00007FF70AFD1500	48:31C0	xor rax,rax	checker
00007FF70AFD1503	48:F7D0	not rax	
00007FF70AFD1506	C3	ret	
00007FF70AFD1507	6C	insb	
00007FF70AFD1508	24 20	and al,20	

이렇게 늘 1 즉 참만 반환하게 만들면

```

D:\source\security\license_project\x64\Release\license_project_v2.exe
Name: Tom
Serial: 03E1-F27A-8CDD-7EA5
Correct!

```

이렇게 잘못된 입력(올바른 입력은 03E1-F27A-8CDD-7EA7)을 넣어도 CORRECT가 뜬다.

자 이제 두 번째 방식의 바이너리 패치를 해 보자

00007FF70AFD1772	48:8D4D C0	lea rcx,qword ptr ss:[rbp-40]	rcx:NtQi
00007FF70AFD1776	48:8D55 A0	lea rdx,qword ptr ss:[rbp-60]	
00007FF70AFD177A	48:0F45D1	cmovne rdx,rcx	rcx:NtQi
00007FF70AFD1781	48:8D0D 1B1B0000	lea rcx,qword ptr ds:[<"%s"...>]	rcx:NtQi
00007FF70AFD1786	E8 9AF8FFFF	call <license_project_v2.printf>	
00007FF70AFD1788	33C0	xor eax,eax	
00007FF70AFD178C	48:8B4D 60	mov rcx,qword ptr ss:[rbp+60]	main.c:1
00007FF70AFD178F	48:33CC	xor rcx,rsi	rcx:NtQi
00007FF70AFD1794	E8 1C000000	call <license_project_v2.__security_check>	
00007FF70AFD1799	48:81C4 70010000	add rsp,170	

위의 바이너리 패치를 v3으로 만들고 다시 v2로 돌아와서 우선 맨 마지막 printf로 로그인 성공 여부를 알려주는 부분의 주소를 복사해서 위치를 파악하고 위치를 파악한 다음에 그걸 뭘로 바꿀까 생각하던 중 그냥 cmovne의 조건부 변수를 그냥 mov rdx, rcx로 늘 printf가 correct를 출력하게 하려고 했는데 길이가 달라서 애러가 떠 nop을 하나 더 추가해줬다 그랬더니

00007FF70AFD1772	48:8D55 A0	lea rdx,qword ptr ss:[rbp-60]	rcx:NtQi
00007FF70AFD1776	48:89CA	mov rdx,rcx	
00007FF70AFD1779	90	nop	
00007FF70AFD177A	48:8D0D 1B1B0000	lea rcx,qword ptr ds:[<"%s"...>]	rcx:NtQi
00007FF70AFD1781	E8 9AF8FFFF	call <license_project_v2.printf>	
00007FF70AFD1786	33C0	xor eax,eax	
00007FF70AFD1788	48:8B4D 60	mov rcx,qword ptr ss:[rbp+60]	main.c:1
00007FF70AFD178C	48:33CC	xor rcx,rsi	rcx:NtQi

```

D:\source\security\license_project\x64\Release\license_project_v2.exe
Name: Tom
Serial: sadsadsa
Correct!

```

이렇게 serial값을 이상하게 넣어도 로그인이 되는 걸 확인할 수 있다.

2.7 배운점

기존 리버싱 project1에서는 매우 간단하고 단순한 기초적인 리버싱 방법에 대해서 학습하고 기본적인 리버싱 툴을 다루는 법 위주로 학습을 했다면 이번에는 좀 더 심화된(암호화,가짜 검증 함수,조금 더 복잡한 매커니즘,디버거 방지 프로그램)등이 있는 조금 더 복잡한 리버싱을 하면서 복잡한 리버싱을 해결하기 위해서는 단순히 ida에 나온 디컴파일된 내용만 믿을 게 아니라 어셈블리어를 보고 실제로 이 함수가 어떤 기능을 하는지를 파악하고 가짜 검증함수가 뭔지를 파악하고 함수의 기능을 토대로 이름을 재설정하고 너무 복잡한 함수의 경우에는 동적분석과 정적분석을 둘 다 활용해 그것이 어떤 기능을 하는지를 우선 정적 함수로 대략 파악한 후에 동적분석을 통해서 그게 실제로 어떻게 레지스터값을 변화시키는지를 파악하면서 함수의 기능을 파악하고 조합해서 그 최종기능을 파이썬으로 구현한 다음에 그것을 토대로 정답을 만들어 내는 입력을 만들어 내는 과정 전체를 배웠고 그 외에도 string table을 보는 법 ida의 데이터값을 xdbg64에서 보는 법 등 다양한 내용에 대해 배우면서 리버싱 실력이 향상된 거 같다.

2.8한계점

분명히 지난번 project1에 비해서 많이 복잡해지고 심화되긴 했지만 여전히 단일 프로그램이고 비교적으로 간단한 편이여서 더 복잡하고 암호 보완이 심화된 프로그램에 대해서도 리버싱을 통한 분석이 필요하고 어셈블리어 해석 능력을 기르기 위해 sub project를 lab 실습 느낌으로 한 2,3개 정도 진행해야 할 필요성을 느꼈다.